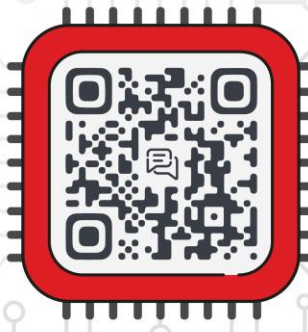




2



15



1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Control Unit Design

(2)

د. سيرا أستور



RB Informatics; 20/10/2025

بنيان الحواسيب 2

بسم الله الرحمن الرحيم

أفكار المحاضرة الرئيسية:

1. طرق بناء وحدة التحكم داخل المعالج والمقارنة بينهما
2. مفهوم Microprogramming
3. مفهوم Microinstruction Encoding مع مقارنة بينهما
4. أمثلة عملية

نبدأ بمراجعة اهم الأفكار التي تحدثنا عنها في المحاضرة السابقة:

- كل معالج مصمم ليعمل بعدد محدد من المعاملات (Operands) لكل تعليمة. على سبيل المثال، في معمارية MIPS، تستخدم العديد من التعليمات ثلاث معاملات (Two-source, one-destination)، بينما في معمارية x86 من Intel، تستخدم الكثير من التعليمات معاملتين فقط.
- إذا كان لدينا أكثر من دخل فيمكننا استخدام MULTIPLEXER (Mux) بحيث يكون الخرج دائماً واحد

طرق تصميم وحدة التحكم:

1. Hard-wired (hardware):

أي أن إشارات التحكم تتولد مباشرة من الدارات المنطقية، أي في حال أردنا تغيير سلوك وحدة التحكم يلزمنا تغيير الدارات المنطقية، وتبقى المعادلة نفسها. نجد تطبيقه في (ROM) و (PLA)

مثلاً: في حال إضافة تعليمة جديدة إلى مجموعة التعليمات (Instruction Set)، يجب تعديل أو إعادة تصميم الدوائر المنطقية بالكامل، مما يجعلها غير مرنة.

المزايا: سريعة جداً لأنها تعمل مباشرة على الإشارات

العيوب: غير مرنة (صعبة التعديل)

2. Firmware (Microprogrammed Control):

هنا، يتم تخزين تسلسل إشارات التحكم اللازمة لتنفيذ كل تعليمة كـ "برنامج صغير" (Microprogram) في ذاكرة خاصة سريعة (Microprogram Memory). تعمل وحدة تحكم خاصة على تنفيذ هذه التعليمات الدقيقة (Microinstructions) واحدة تلو الأخرى، كما أنها مبنية بمنطقة ثابتة (دارات كهربائية) غير قابلة للتعديل.

المزايا:

- مرنة: فبدلاً من تغيير الدارات المنطقية نستطيع تغيير البرنامج داخل الذاكرة التي فيها الـ Micro-program
- قابلة للتقليد والتبديل: فمثلاً بعض المعالجات القديمة فقط تم تغيير الـ Micro-program فظهر وكأنه نوع آخر من المعالجات

العيوب:

السرعة المحدودة: لأن وحدة التحكم يجب أن تقرأ الـ Micro-instruction من الذاكرة الداخلية قبل تنفيذها (تصبح أبطأ)
و الـ (firmware) هو البرنامج الداخلي للجهاز ويكون مضمن داخل الهاردوير نفسها عادة يكون داخل الـ ROM ويكون مسؤول عن تشغيل الجهاز نفسه وغالباً هذا البرنامج لا يتغير إلا عندما تقوم الشركة المصنعة بتحديثه من جديد

أمثلة:

- BIOS : داخل الـ ROM
- Firmware : الموجود في الغسالة الذي يتحكم بالموتور ودورة الغسيل

مثال عن تصميم وحدة التحكم عن طريق الـ Hardware Design:

بفرض لدينا معالج بسيط، (غير معالج MIPS) فقط معمارية بسيطة، ولديه فقط 4 تعليمات وهي:

- $ADD\ x1$: أضف القيمة من العنوان $x1$ إلى الـ Accumulator
- $AND\ x1$: طبق العملية AND بين القيمة في العنوان $x1$ والـ Accumulator
- $JMP\ x1$: اقفز إلى العنوان $x1$ (غير قيمة الـ PC إلى القيمة الموجودة في العنوان $x1$).
- INC : زد الـ Accumulator بمقدار 1.

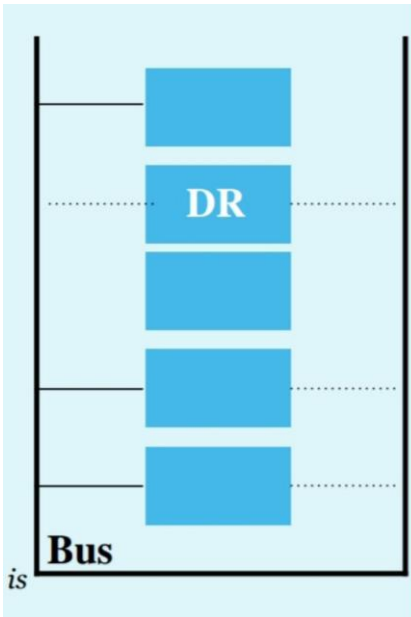
نلاحظ أن جميع التعليمات فيها معامل واحد فقط والمعامل الثان

ي هو Accumulator (سجل خاص داخل المعالج يستخدم لتخزين النتائج المؤقتة) أي جميع العمليات مثل ADD تنفذ على قيم AC مباشرة.

- $ADD\ x1 \rightarrow AC = AC + Memory[x1]$
- $AND\ x1 \rightarrow AC = AC AND Memory[x1]$
- $INC \rightarrow AC = AC + 1$

ولدينا باقي السجلات:

AR	Address Register	يحتوي عنوان الذاكرة لأي موقع تتم قراءته أو الكتابة عليه
DR	Data Register	يحتوي المعطيات التي تتم قراءتها/كتابتها في الذاكرة
IR	Instruction Register	يخزن التعليمة الحالية التي نريد تنفيذها
AC	Accumulator Register	يخزن النتائج المؤقتة أو النهائية للعملية

**والمخطط يوضح عملية تبادل البيانات بين السجلات عن طريق الـ Bus فمثلاً:**

يمكن الـ AR يرسل العنوان إلى الذاكرة عبر الباص
والذاكرة ترجع البيانات إلى DR عبر نفس الباص
والـ AC يستخدم الباص لاستلام البيانات أو إرسالها

ملاحظة:

بما أن كل تعليمة تحتوي على معامل واحد فقط هو $x1$ و المعامل الثاني هو AC فإن عملية فك التشفير ستكون سهلة وهي عبارة عن قراءة الـ Opcode (نوع العملية) + الـ Operand (العنوان)

Example:

- $F1: AR \leftarrow PC$
- $F2: DR \leftarrow M; PC \leftarrow PC + 1$
- $F3: IR \leftarrow DR(OPERAND); AR \leftarrow DR(OPERAND) ADD1: DR \leftarrow M$
- $ADD2: AC \leftarrow AC + DR$
- $AND1: DR \leftarrow M$
- $AND2: AC \leftarrow AC \cdot DR$
- $INC1: AC \leftarrow AC + 1 JMP1: PC \leftarrow DR(OPERAND); \#(OPERAND = X1)$

في هذا المثال سنتعلم كيف الـ ISA سيتصرف عندما تأتي تعليمة جديدة

نعلم أنه كل تعليمة تمر ب 5 مراحل عامة مهما كان نوع المعالج وهي Instruction Cycle:

1. **Fetch**: اجلب التعليمة من الذاكرة
2. **Decode**: معرفة نوع التعليمة
3. **Execute**: نفذ العملية المطلوبة (جمع، قفز، ..)

المرحلة الأولى: وهي Fetch

- الـ AR (Address Register) يأخذ عنوان التعليمة من الـ PC عداد البرنامج
- نستخدم العنوان لقراءة التعليمة من الذاكرة (Memory $\rightarrow$ DR)، وب نفس الوقت نزيد قيمة الـ PC $PC+1 \leftarrow PC$

■ **نسخ محتوى الـ (DR) حيث:** $IR \leftarrow DR(opcode)$ و $AR \leftarrow DR(operand)$

المرحلة الثانية: Decode - فك التعليم

بعد ما دخلت التعليم على الـ IR، يقرأ المعالج الـ Opcode (نوع التعليم) والـ Operand (العنوان أو القيمة التي سنستخدمها)

- المعالج يقرر: إذا كانت $ADD \leftarrow$ نفذ عملية جمع.
- إذا كانت $AND \leftarrow$ نفذ AND
- إذا كانت $JMP \leftarrow$ غير العنوان
- إذا كانت $INC \leftarrow$ زد الـ AC بواحد

المرحلة الثالثة: Execute - تنفيذ التعليم

كل تعليمية من التعليمات الأربعة تنفذ على شكل خطوات صغيرة (micro-operations):

1. $ADD\ x1$

$$AC \leftarrow AC + M[x1]$$

- الـ AR يأخذ العنوان $x1$
- نقرأ من الذاكرة القيمة الموجودة فيه إلى الـ DR أي: $DR \leftarrow M[AR]$
- نجمع بين الـ AC و DR ونخزن النتيجة في الـ AC

2. $AND\ x1$

$$AC \leftarrow AC\ AND\ M[x1]$$

- نفس الفكرة، بدل الجمع، ننفذ عملية AND منطقية.

3. $JMP\ x1$

$$PC \leftarrow x1$$

- هنا المعالج لا يستخدم الـ AC، فقط يغير عداد البرنامج (PC) إلى العنوان $x1$ ، أي يقفز لمكان آخر

4. INC

$$AC \leftarrow AC + 1$$

- المعالج يزيد قيمة الـ AC بمقدار 1

Step 2: State Diagram

- $F1: AR \leftarrow PC$
- $F2: DR \leftarrow M; PC \leftarrow PC + 1$
- $F3: IR \leftarrow DR(opcode); AR \leftarrow DR(x1)$
- $Add1: DR \leftarrow M$
- $Add2: AC \leftarrow AC + DR$

- $And1: DR \leftarrow M$
- $And2: AC \leftarrow AC . DR$
- $Inc1: AC \leftarrow AC + 1$
- $Jump1: PC \leftarrow DR(x1)$

الحالة F1 (جلب العنوان - Fetch Address)

العملية: $AR \leftarrow PC$

الشرح: هنا يقوم المعالج بوضع عنوان التعليمة التالية (المخزن في عداد البرنامج PC) على "عنوان الناقل" (Address Bus) ليخبر الذاكرة بأي عنوان يريد قراءته. يتم ذلك عن طريق نسخ قيمة PC إلى AR (Address Register)

الحالة F2 (جلب التعليمة - Fetch Instruction)

العملية: $DR \leftarrow M; PC \leftarrow PC + 1$

الشرح: في هذه الحالة تقوم الذاكرة بوضع محتويات العنوان المطلوب (الموجود في AR) على "بيانات الناقل" (Data Bus)، يقوم المعالج (Processor) بقراءة هذه البيانات (وهي التعليمة) وتخزينها في DR (Data Register) في نفس الوقت، يزيد المعالج قيمة PC بواحد ليصبح مشيراً إلى التعليمة التالية في البرنامج، استعداداً لدورة الجلب القادمة

الحالة F3 (فك تشفير التعليمة - Decode Instruction)

العملية: $IR \leftarrow DR(opcode); AR \leftarrow DR(operand)$ الشرح: التعليمة المخزنة في DR تنقسم إلى جزأين:▪ رمز العملية (Opcode): يحدد العملية المطلوب تنفيذها (جمع، منطق، قفز، إلخ)▪ المعامل (Operand): هو عنوان البيانات (في الذاكرة) التي ستطبق عليها العملية

هنا، يتم نسخ جزء Opcode من DR إلى IR (Instruction Register)، حيث تستخدمه وحدة التحكم لمعرفة أي عملية مطلوبة.

في نفس الوقت، يتم نسخ جزء Operand (العنوان) من DR إلى AR، ليصبح جاهزاً لاستخدامه في جلب البيانات من الذاكرة في الخطوة التالية إذا لزم الأمر.

بعد F3، تبدأ مرحلة التنفيذ، وتعتمد الخطوات التالية على التعليمة التي تم فك تشفيرها:

لتعليمة الجمع (ADD):

الحالة ADD1: $DR \leftarrow M$

الشرح: المعالج يطلب من الذاكرة البيانات الموجودة في العنوان المخزن في AR، تقوم الذاكرة بإرجاع البيانات (مثلاً: الرقم 5) ويتم تخزينها في DR.

الحالة ADD2: $AC \leftarrow AC + DR$

الشرح: هذه هي خطوة التنفيذ الفعلية يتم جلب قيمة المسجل (المتراكم) AC وإضافتها إلى القيمة الموجودة في DR. ثم يتم تخزين الناتج مرة أخرى في AC، بعد هذه الخطوة، تعود دورة المعالج إلى حالة البداية F1.

لتعليمة الضرب المنطقي (AND):

الحالة AND1: $DR \leftarrow M$

الشرح: نفس خطوة ADD1 تماماً، يتم جلب البيانات من الذاكرة وتخزينها في DR.

الحالة AND2: $AC \leftarrow AC . DR$

الشرح: هنا يتم تنفيذ عملية الضرب المنطقي (AND) بين قيمة AC وقيمة DR. الناتج يُخزن في AC. ثم العودة إلى F1.

لتعليمة الزيادة (INC):

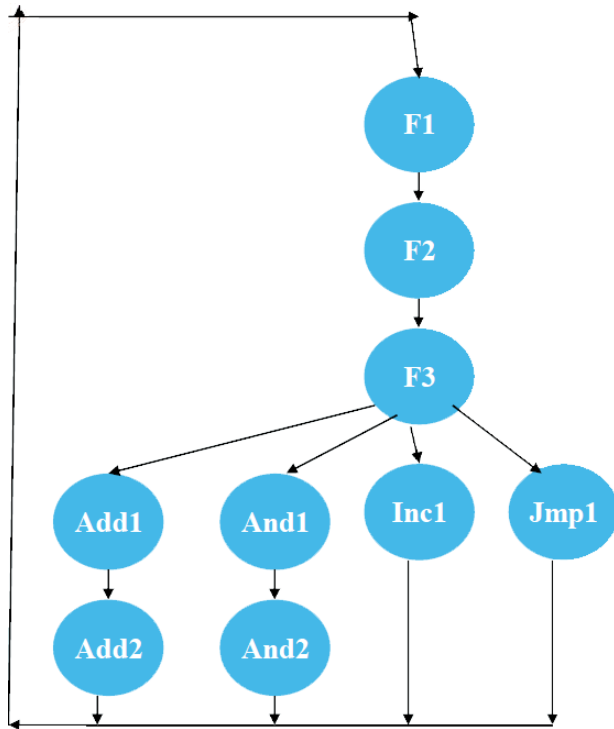
الحالة INC1: $AC \leftarrow AC + 1$

الشرح: هذه تعليمة لا تحتاج للوصول للذاكرة بمجرد فك التشفير في F3، تنتقل وحدة التحكم مباشرة إلى هذه الحالة لزيادة قيمة المسجل AC بواحد، ثم العودة إلى F1.

لتعليمة القفز (JMP):

الحالة JMP1: $PC \leftarrow DR(operand)$

هذه التعليمة تغير تسلسل تنفيذ البرنامج، بدلاً من جلب البيانات، يتم نسخ قيمة المعامل (operand أو X1) التي كانت مخزنة بالفعل في DR (من مرحلة F2) إلى PC، هذا يجعل PC يشير إلى عنوان جديد في الذاكرة، وبالتالي ستجلب التعليمة التالية من ذلك العنوان الجديد في دورة الجلب القادمة (F1).



Step 3: Design Datapaths & Specify the Control Signals:

- $F1: AR \leftarrow PC$
- $F2: DR \leftarrow M; PC \leftarrow PC + 1$
- $F3: IR \leftarrow DR(opcode); AR \leftarrow DR(x1)$
- $Add1: DR \leftarrow M$
- $Add2: AC \leftarrow AC + DR$

- $And1: DR \leftarrow M$
- $And2: AC \leftarrow AC \cdot DR$
- $Inc1: AC \leftarrow AC + 1$
- $Jmp1: PC \leftarrow DR(x1)$

خطوات تصميم وحدة التحكم والمسارات :

أولاً: إشارات التحكم (Control Signals):

1. ARLD: تحميل سجل العنوان
2. DRLD: تحميل سجل البيانات
3. IRLD: تحميل سجل التعليمات
4. Read: قراءة من الذاكرة
5. Mem: تفعيل الذاكرة
6. Mem2bus: نقل البيانات من الذاكرة إلى الناقل

العمليات المحددة:

(Fetch Cycle):

1. $F1: AR \leftarrow PC$: نقل عنوان من عداد البرنامج إلى سجل العنوان
2. $F2: DR \leftarrow M; PC \leftarrow PC + 1$: قراءة التعليم من الذاكرة وزيادة عداد البرنامج
3. $F3: IR \leftarrow DR(opcode); AR \leftarrow DR(x1)$: تحميل كود العملية وتحديد العنوان

عمليات التنفيذ:

1. ADD: جمع ($AC \leftarrow AC + DR$)
2. AND: منطقية AND عملية ($AC \leftarrow AC . DR$)
3. INC: زيادة ($AC \leftarrow AC + 1$)

Step 4: Implementing Control Signals:

كيف يتم توليد إشارات التحكم باستخدام معادلات منطقية تعتمد على حالة التعليمات الحالية؟

1. تحميل السجل التراكمي (ACLD):

$$ACLD = add2 + inc1 + and2$$

يتم تحميل AC عندما:

1. add2: تنفيذ عملية الجمع
2. inc1: تنفيذ عملية الزيادة
3. and2: تنفيذ عملية AND

2. تحميل سجل العنوان (ARLD):

$$ARLD = F1 + F3$$

يتم تحميل AR في:

1. $F1: AR \leftarrow PC$ جلب التعليم
2. $F3: AR \leftarrow DR(x1)$ فك الشفرة

3. تحميل سجل البيانات (DRLD):

$$DRLD = F2 + add1 + and1$$

يتم تحميل DR في:

1. $F2$: جلب التعليم من الذاكرة
2. add1: مرحلة الجمع الأولى
3. and1: مرحلة AND الأولى

4. التحكم في الناقل (Bus Control):

$$DRabus = jmp1 + and2 + add2 + F3$$

الدور: يتحكم في نقل البيانات من DR إلى الناقل (Bus)

5. تحميل سجل التعليمات (IRLD):

$$IRLD = F3$$

الدور: تحميل كود العملية F3 في سجل التعليمات

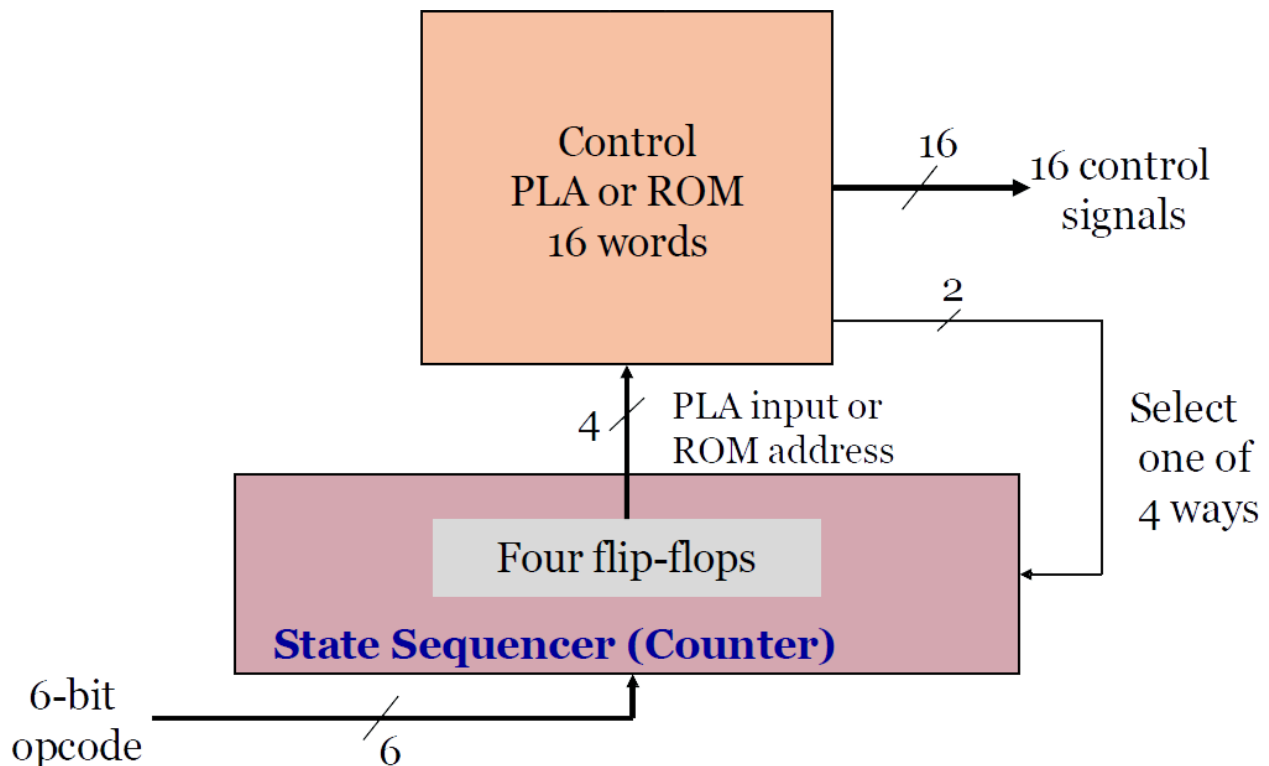
6. التحكم في (PC):

$$\begin{aligned} PC2bus &= F1 \\ PCLD &= jmp1 + F2 \\ PCINC &= F2 \end{aligned}$$

الدور:

4. PC2bus: نقل PC إلى الناقل
5. PCLD: يحمل قيمة جديدة في PC
6. PCINC: يزيد PC بمقدار 1

Implementing with ROM/PLA:

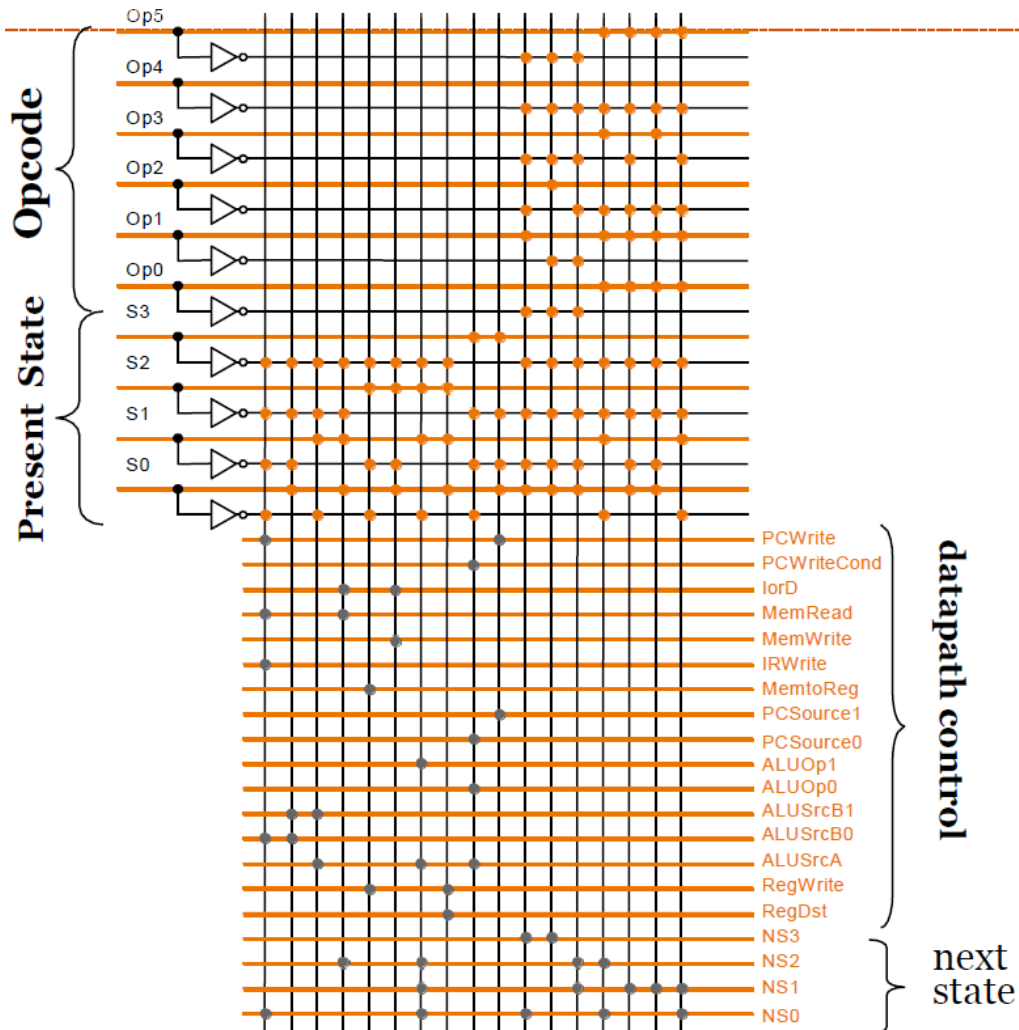


- هذا المخطط يوضح طريقة تنفيذ وحدة تحكم Datapath باستخدام PLA/ROM
- بدلاً من بناء شبكة AND/OR، نستخدم ذاكرة ROM مخزنة فيها «كلمات ميكرو تعليمية» (microinstructions).

- عنوان هذه الذاكرة يُشتق من الحالة الحالية (أو من الحالة + أجزاء من الـ opcode حسب التصميم)، وكل كلمة في ROM تحوي قيمة جميع إشارات التحكم.
- في الرسم مكتوب «16 words» — هذا يعني أن العنوان مكوّن من 4 بت ($4^2 = 16$) في أبسط استخدام: أي لكل حالة من الأربع فليب-فلوب الأربعة هناك كلمة في ROM.
- الـ State Sequencer (العداد ذو الفليب-فلوب الأربعة) يمثل عدد خطوات الميكرو (مثلاً T0... T15)، قيمته تُستخدم كعنوان لإخراج كلمة تحكم من الـ ROM أو كمداخل للـ PLA.
- أحياناً نحتاج أن تتأثر إشارات التحكم بنوع التعليم (opcode)، لذلك يوجد اتصال من الـ opcode إلى PLA (أو تؤخذ أجزاء منه لتحديد أي "way" أو مجموعة حروف في ROM).
- المخرجات من الـ PLA/ROM (16 إشارة تحكم في المخطط) تُرسل إلى Datapath لتنفيذ عمليات مثل قراءة/كتابة الذاكرة، تحميل سجل، تشغيل الـ ALU، تحديث عدّاد البرنامج، إلخ.

ملاحظة على السطور الصغيرة (2/4):

الرسم يبيّن أيضاً خطوطاً عرضها 2 و 4 وبالتحديد «Select one of 4 ways» — هذا يعني احتمال وجود تقسيم داخلي: العنوان إلى الـ ROM قد ينسب من (4 بت حالة) + (2 بت اختيار تؤخذ من opcode) فتنتج 16 موقعاً = 4 طرق × 4 حالات. لكن المبدأ لا يتغيّر: عنوان → كلمة تحكم.



الأعمدة التي على اليسار:

- صفوف تمثل إشارات الإدخال وهنا تظهر خطوط للـ opcode (op5..op0) وأيضًا خطوط للحالة الحالية (S3..S0).
- بجانب كل خط رمز عاكس (inverter) لأن الـ PLA عادة يسمح باستخدام إشارة موجبة أو سالبة.

الشبكة الكبيرة:

- نقاط (أو وصلات) تشير إلى أماكن يتم فيها توصيل كل إشارة (أو نفيها) مع خطوط عمودية تمثل terms

على اليمين:

- أعمدة الخرج (كل عمود عبارة عن إشارة تحكم أو بت حالة التالية)، كل عمود يتجمع عنده عدد من الـ product-terms (نقاط سوداء أو برتقالية) ثم تمر عبر OR لتنتج إشارة الخرج.

كيف تقرأ الشبكة عملياً (خطوة-خطوة):

- انظر للحالة الحالية S3..S0 والـ opcode op5..op0 – هذه هي المدخلات.
- كل عمود في شبكة الـ AND تمثل ناتج (AND) من بعض هذه المدخلات أو من نفيها.
- **مثال:** إذا كان لدينا نقطة عند تقاطع op2 (غير معكوس) و S1 (معكوس) و S0 (غير معكوس) فهذا المنتج يكون: op2 AND (NOT S1) AND S0.
- كل إشارة تحكم على اليمين تكون مجموع OR لهذه النواتج أي إشارة تحكم = ProdTerm1 OR ... ProdTerm2 OR
- كذلك يتم إنتاج بتات الحالة التالية (NS3..NS0) بنفس الطريقة — المخرجات التي تمثل الحالة التالية تغذي الفليب-فلوب في حالة التسلسل التالي.

Micro-Program

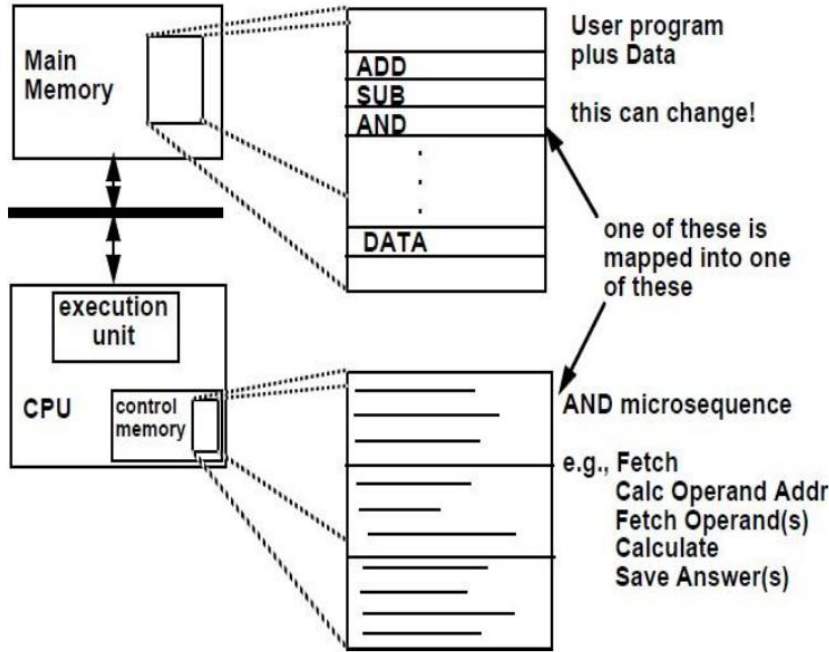
(Micro-Instruction): كل أمر رئيسي (مثل ADD) لا يتم تنفيذه في خطوة واحدة، بل يقسم إلى سلسلة من الخطوات الأولية الأصغر مثل:

- جلب الأمر من الذاكرة (Fetch)
- فك شيفرة الأمر (Decode)
- حساب عنوان المعطى (Calculate Operand Address)
- جلب المعطى نفسه (Fetch Operand)
- تنفيذ العملية الحسابية الفعلية (Execute)
- حفظ النتيجة (Save Result)

أين يتم التخزين؟

يتم تخزينها في ذاكرة خاصة سريعة جداً داخل وحدة التحكم نفسها، تسمى الذاكرة الدقيقة، محتويات هذه الذاكرة ثابتة ولا تتغير بواسطة برامج المستخدم العادية (على عكس الذاكرة الرئيسية)، هي بمثابة "BIOS" أو نظام التشغيل الأساسي جداً للمعالج نفسه.

Macroinstruction Interpretation



ماذا يمثل الرسم؟

- **الجزء العلوي (Main Memory):** يمثل ذاكرة الحاسوب الرئيسية، التي تحتوي على برنامج المستخدم (مجموعة من الأوامر الرئيسية مثل ADD, SUB, AND) ومعطياته و هذه الذاكرة قابلة للتغيير.
- **الجزء السفلي (Control Memory):** يمثل ذاكرة التحكم الداخلية في المعالج، محتواها ثابت ويحدد كيفية تنفيذ كل أمر رئيسي.
- **السهم (Mapping):** يوضح العلاقة: عندما يقرأ المعالج أمراً رئيسياً من الذاكرة (مثل أمر ADD للجمع)، فإن

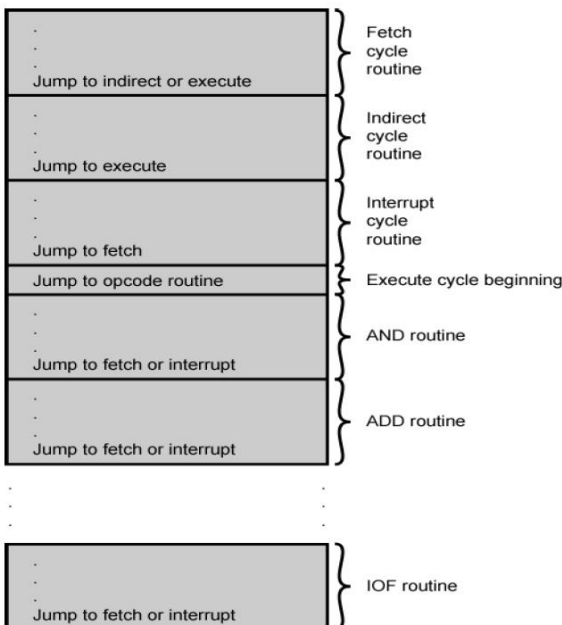
هذا الأمر هو مجرد رمز يخبر المعالج ماذا يفعل وبذلك فإن Control Memory لا يحتوي على أمر ADD مكتوباً بشكل صريح، بل يحتوي الى مجموعة من الخطوات البسيطة جداً Micro-instruction التي يجب تنفيذها بالتسلسل لإكمال عملية الجمع.

■ مثال توضيحي:

عندما يصادف المعالج الأمر ADD بداخله يبدأ Micro-Program بتنفيذ البرنامج المخزن في العنوان المخصص له في الذاكرة ، كالتالي:

1. انتقل إلى روتين "جلب الأمر" (Fetch).

2. انتقل إلى روتين "فك الشيفرة" (Decode).



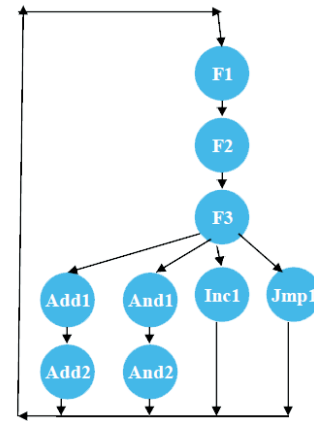
3. انتقل إلى روتين "حساب عنوان المعطى".
4. انتقل إلى روتين "جلب المعطى".
5. انتقل إلى روتين تنفيذ عملية الجمع.
6. انتقل إلى روتين "حفظ النتيجة".
7. أخيراً، ارجع إلى بداية دورة الجلب لاستلام الأمر التالي.

لنتابع تتمة شرح المثال الأخير:

● Step 1: Define the micro-instructions

$F1: AR \leftarrow PC$
 $F2: DR \leftarrow M; PC \leftarrow PC+1$
 $F3: IR \leftarrow DR(opcode); AR \leftarrow DR(x1)$
 $Add1: DR \leftarrow M$
 $Add2: AC \leftarrow AC+DR$
 $And1: DR \leftarrow M$
 $And2: AC \leftarrow AC.DR$
 $Inc1: AC \leftarrow AC+1$
 $Jump1: PC \leftarrow DR(x1)$

ARDR
 IRDR
 INCPC
 DRM
 ARPC
 ACaddDR
 ACINC
 PCDR
 ACandDR



الخطوة 2: تنظيم التعليمات الذاكرة الدقيقة (Micro-Memory)

كل موقع في هذه الذاكرة يخزن (Micro-Word) تحتوي على جزأين رئيسيين:

1. مجال إشارات التحكم / العمليات الدقيقة (Control Field / UOP): يحدد أي من العمليات الدقيقة، مثل: $F1$ $Add2$ (إلخ) سيتم تنفيذها في هذه الدورة.
2. مجال العنوان التالي (Next Address Field): يحدد عنوان الكلمة الدقيقة التالية التي يجب تنفيذها وهذا ما ينشئ التسلسل للبرنامج

F1 (العنوان 0):

- $001 \rightarrow 0$: العنوان التالي هو 1 (تسلسلي).
- 100000000 : هذا النمط من البتات ينشط إشارة التحكم الخاصة بالعملية الدقيقة $F1$ فقط ($AR \leftarrow PC$).

F2 (العنوان 1):

- $0010 \rightarrow 0$: العنوان التالي هو 2 (تسلسلي).
- 011000000 : هذا النمط ينشط إشارتي تحكم معاً ($INCPC$ و DRM) لتنفيذ $DR \leftarrow M$ و $PC \leftarrow PC+1$ في نفس الوقت.

F3 (العنوان 2):

1 → xxxx: هنا يتم اختيار العنوان التالي بناءً على حالة خارجية (مثل قيمة opcode في IR)، الـ xxxx تعني أن العنوان التالي سيعتمد على كود العملية، إذا كان الكود ADD، قد يذهب إلى عنوان Add1، وإذا كان AND يذهب إلى And1، وهكذا.

000110000: ينشط إشارات التحكم الخاصة بـ $IR \leftarrow DR(opcode)$ و $AR \leftarrow DR(address)$.
وحدة الـ LOGIC تولّد إشارات تحكم (Control signals) تتحكم في مكونات المعالج مثل السجلات (registers) وذاكرة البيانات (DRM).
وكل معادلة تمثل عملية ميكروية (Micro-operation)، أي:

المعنى	المعادلة المختصرة	العملية في الخطوات
نقل قيمة عدّاد البرنامج (PC) إلى مسجل العنوان (AR)	ARPC	F1: $AR \leftarrow PC$
الأولى تقرأ من الذاكرة إلى DR، والثانية تزيد PC بواحد	DRM و INCPC	F2: $DR \leftarrow M; PC \leftarrow PC + 1$
الأولى تنسخ كود العملية من DR إلى IR، والثانية تنقل العنوان من DR إلى AR	ARDR و IRDR	F3: $IR \leftarrow DR(opcode); AR \leftarrow DR(address)$
تحميل القيمة من الذاكرة إلى DR	DRM	Add1: $DR \leftarrow M$
جمع محتويات المسجلين AC و DR	ACaddDR	Add2: $AC \leftarrow AC + DR$
نفس العملية (قراءة من الذاكرة)	DRM	And1: $DR \leftarrow M$
تنفيذ عملية AND بين AC و DR	ACandDR	And2: $AC \leftarrow AC \wedge DR$
زيادة محتوى AC بواحد	ACINC	Inc1: $AC \leftarrow AC + 1$
تحميل قيمة من DR إلى عدّاد البرنامج PC	PCDR	Jmp1: $PC \leftarrow DR(address)$

طرق ترميز إشارات التحكم (Encoding Types):

1. الترميز المباشر (Direct Encoding):

المبدأ: كل إشارة تحكم (تشغيل وحدة الجمع في الـ ALU) لها بت خاص بها في كلمة التحكم.
الميزة: السرعة حيث لا حاجة لفك ترميز (Decoding) الإشارات تنشيط مباشرة. متوازي بالكامل.
العيوب: هدر كبير في المساحة.

2. الترميز الأفقي (Horizontal Encoding):

المبدأ: كل عملية صغيرة لها بت منفصل.

3. الترميز العمودي (Vertical Encoding):

المبدأ: بدلاً من تخصيص بت لكل إشارة، نقوم بتجميع الإشارات في حقول (Fields)، كل حقل يمثل مجموعة من الإشارات المتنافية (لا يمكن تنشيطها معاً)، مثلاً: حقل "مصدر بيانات الـ ALU" يمكن أن يكون 2 بت ويمثل 4 خيارات (مثل 00 من 01، AC من 10، DR من 11، M من PC).

Micro-Programming Example:

المعطيات:

عدد إشارات التحكم (Control Signals) = 120

عدد العمليات الميكروية (Micro-operations) = 63

عدد التعليمات الميكروية (Micro-instructions) = 29

ما حجم الذاكرة المطلوبة لتخزين هذه التعليمات الميكروية باستخدام:

- Direct Encoding
- Horizontal Encoding
- Vertical Encoding (إذا أمكن تقسيم التعليمات إلى مجموعتين)

الحل:

عدد التعليمات الميكروية = 29

يعني نحتاج 5 bits لتمثيلها (لأن $2^5 = 32$ أي تكفي من أجل 29 تعليمة)
إذن 5 بت تستخدم ك عنوان (address).

1. Direct Encoding:

في الترميز المباشر، كل إشارة تحكم (Control Signal) تمثل بت واحد في التعليمة الميكروية.

عدد إشارات التحكم = 120

ونضيف 5 بت للعنوان و 1 بت من أجل next-address أو condition bit

إجمالي البتات = $120 + 5 + 1 = 126$ bits

حجم الذاكرة = 29 كلمة (Word)

أي 29×126

كل كلمة طولها 126 بت

2. Horizontal Encoding:

في الترميز الأفقي، نجمع إشارات التحكم في مجموعات تُفعّل فيها أكثر من عملية معاً، مما يقلل عدد البتات.

عدد العمليات الميكروية = 63

نضيف 5 بت للعنوان + 1 بت للتحكم

إجمالي البتات = $63 + 5 + 1 = 69$ بتحجم الذاكرة = 29×69

كل كلمة طولها 69 بت

3. Vertical Encoding:

في Vertical Encoding نحاول تقليل عدد البتات في كل micro-instruction عن طريق تقسيم الميكرو-عمليات إلى مجموعات، بحيث في كل لحظة لا تُنفَّذ إلا عملية واحدة من كل مجموعة. يعني قسمنا الميكرو عمليات (63) إلى مجموعتين تقريباً متساويتين: كل مجموعة فيها حوالي 32 عملية. لكي نمثل 32 عملية مختلفة في مجموعة واحدة، نحتاج: 5 بت

إِذَا:

المجموعة الأولى → 5 بت

المجموعة الثانية → 5 بت أيضاً

المجموع = 10 بت

لكن لأن العملية "Null" (بدون تنفيذ) ممكنة، نحتاج إضافة 1 بت إضافي لتحديد إن كانت هناك عملية أو لا.

Total bits = (5+5+1) = 11 bits

Memory size = 29 × 11 bits



THE END ^ _ ^